# Space complexity: L, PSPACE, and NL

Exercise sheet 11 (solutions)

**Logspace.**

**Exercise 11.1.** Let $A$, $B$, and $C$ be languages. Prove the following statements about logspace reductions:

(a) If $A \leq_L B$ and $B \leq_L C$, then $A \leq_L C$.

(b) If $A \leq_L B$ and $B \in \mathrm{L}$, then $A \in \mathrm{L}$.

(c) Suppose that $B$ is P-complete under logspace reductions and $B \in \mathrm{L}$. Prove that $\mathrm{L} = \mathrm{P}$.

*Solution.* Write $r_1$ and $r_2$ for the two reductions in part (a). Their composition preserves membership. To compute it in logarithmic space, simulate the transducer for $r_2$ on the virtual input $r_1(x)$. Whenever it asks for the symbol at a particular input position, rerun the transducer for $r_1$ on $x$ and count its output symbols until that position is reached. The output of $r_1$ has polynomial length, so its position counter uses $O(\log |x|)$ bits. The simulation uses logarithmic space and computes a logspace reduction from $A$ to $C$.

For part (b), run the logspace decider for $B$ on the virtual input $r(x)$ in the same way. It accepts exactly when $r(x) \in B$, which holds exactly when $x \in A$. Thus $A \in \mathrm{L}$.

For part (c), take any $A \in \mathrm{P}$. Since $B$ is P-complete, $A \leq_L B$. Part (b) then gives $A \in \mathrm{L}$. Hence $\mathrm{P} \subseteq \mathrm{L}$; together with $\mathrm{L} \subseteq \mathrm{P}$, this gives equality. □

**Exercise 11.2.** Prove that L is closed under union and intersection: if $A, B \in \mathrm{L}$, then $A \cup B \in \mathrm{L}$ and $A \cap B \in \mathrm{L}$.

*Solution.* Run the two deterministic logspace deciders one after the other, reusing the same work tape. Accept if either accepts to decide $A \cup B$; accept if both accept to decide $A \cap B$. The combined machine still uses logarithmic space. □

**Exercise 11.3.** A word over $\{(,)\}$ is a balanced-parentheses word if every prefix contains at least as many left parentheses as right parentheses, and the whole word contains equal numbers of each. Prove that the language of balanced-parentheses words is in L.

*Solution.* Scan the input from left to right, keeping a counter $d$ for the number of left parentheses minus the number of right parentheses seen so far. Increase it on '(', and decrease it on ')'. Reject if it ever becomes negative, and accept at the end exactly when it is zero. The counter is between 0 and the input length, so it uses $O(\log n)$ bits. This algorithm checks both the prefix condition and the equality of the total counts. □

**Polynomial space.**

**Exercise 11.4.** A fully quantified Boolean formula is true if its quantified variables can be assigned values according to the quantifiers so that the formula evaluates to true. Give an algorithm that decides this problem using polynomial space. Explain why it may take much more than polynomial time.

*Solution.* Evaluate the quantifiers recursively. For an existentially quantified variable, evaluate the remaining formula with the variable set to false and with it set to true, accepting if either recursive call accepts. For a universally quantified variable, accept if both calls accept. Once all variables are assigned, evaluate the matrix.

Store the current assignment, the position in the quantifier prefix, and enough information to return from each recursive call. This takes polynomial space in the formula length; when one call returns, reuse its workspace for the next call. The recursion may examine both choices for each of $n$ variables, so it can make up to $2^n$ recursive calls. Thus the procedure uses polynomial space but need not run in polynomial time. □

**Exercise 11.5.** * Let TQBF-CNF be the problem of deciding the truth of a fully quantified Boolean formula whose unquantified matrix is in conjunctive normal form. Prove that TQBF-CNF is PSPACE-complete under polynomial-time reductions.

*Solution.* Membership in PSPACE follows from the polynomial-space algorithm for quantified Boolean formulas.

For hardness, reduce from TQBF, which is PSPACE-complete. Given

$$Q_1 x_1 \cdots Q_n x_n \, \varphi(x_1, \ldots, x_n),$$

introduce a fresh variable for each gate of the matrix $\varphi$. Add clauses expressing that each new variable equals the value of its gate, and add a clause requiring the output gate to be true. The resulting conjunctive-normal-form formula $C(x_1, \ldots, x_n, y_1, \ldots, y_m)$ has polynomial size and satisfies

$$\varphi(x_1, \ldots, x_n) = \text{true} \quad \Longleftrightarrow \quad \exists y_1 \cdots \exists y_m \, C(x_1, \ldots, x_n, y_1, \ldots, y_m).$$

Append the existential quantifiers for the new variables after the original prefix. For each assignment to the original variables, the new variables can satisfy the clauses exactly when the original matrix is true. Therefore the quantified formula's truth is preserved, and the construction is a polynomial-time reduction to TQBF-CNF. This proves PSPACE-hardness. □

**Exercise 11.6.** * Define IN-SPACE-ACCEPTANCE as follows:

**Input:** A deterministic Turing machine $M$ and a word $x$.

**Question:** Does $M$ accept $x$ without ever using a tape cell beyond the first $|x|$ cells?

Prove that IN-SPACE-ACCEPTANCE is PSPACE-complete.

*Solution.* Let $N$ be the length of the encoded pair $(M, x)$. Simulate $M$ while restricting it to the first $|x|$ tape cells. The simulated tape contents and head position use polynomial space in $N$. There are at most

$$|Q|\,(|x|+1)\,|\Gamma|^{|x|}$$

configurations, where $Q$ and $\Gamma$ are the state set and tape alphabet of $M$. The number of bits needed to count this many steps is $O(\log|Q| + \log(|x|+1) + |x|\log|\Gamma|)$, which is polynomial in $N$. If $M$ accepts before the counter runs out, accept; if it rejects, or tries to leave the permitted cells, reject. If the counter runs out without halting, a configuration has repeated, so the deterministic computation will never accept. Thus the problem is in PSPACE.

Now let $A \in$ PSPACE, decided by a fixed machine $M_A$ using at most $p(n)$ tape cells on inputs of length $n$, for a polynomial $p$. Map an input $w$ to a pair $(M', w\#0^{p(|w|)})$, where $M'$ is a fixed machine that reads the prefix before '#' and simulates $M_A$ on that prefix. The padded input has polynomial length and permits at least $p(|w|)$ tape cells. Hence $M'$ accepts within that bound exactly when $w \in A$. This is a polynomial-time reduction from $A$ to IN-SPACE-ACCEPTANCE, proving PSPACE-hardness. □

**Exercise 11.7.** * Use the space hierarchy theorem and padding to prove that P $\neq$ SPACE($n$), where SPACE($n$) is deterministic space bounded by a linear function of the input length.

*Solution.* By the space hierarchy theorem, choose a language $B \in \mathrm{SPACE}(n^2) \setminus \mathrm{SPACE}(n)$. Define its padded version by

$$B' = \{x\#0^{|x|^2} \mid x \in B\}.$$

To decide $B'$, check that the input has this form and then simulate the space-$O(|x|^2)$ decider for $B$ on $x$. Since the padded input has length at least $|x|^2$, this uses linear space in the padded input length. Thus $B' \in \mathrm{SPACE}(n)$.

Suppose for contradiction that P = SPACE($n$). Then $B' \in$ P. Given $x$, construct $x\#0^{|x|^2}$ and run the polynomial-time decider for $B'$. Its running time is polynomial in the padded length, hence polynomial in $|x|$. This decides $B$ in polynomial time, so $B \in$ P = SPACE($n$), contradicting the choice of $B$. Therefore P $\neq$ SPACE($n$). □

**Nondeterminism.** In a nondeterministic computation, a machine can have several possible next moves. Its computation can be viewed as a tree of branches. The machine accepts if at least one branch accepts; a space bound applies to each branch separately.

**Exercise 11.8.** * Let NSPACE (s(n)) denote the class of languages decided by nondeterministic machines using $O(s(n))$ working space. Use the deterministic space hierarchy theorem and Savitch's theorem to prove that there is a language in SPACE($n^3$) that is not in NSPACE (n). You may quote both theorems.

*Solution.* By the deterministic space hierarchy theorem, there is a language $A \in \text{SPACE}(n^3) \setminus \text{SPACE}(n^2)$. Savitch's theorem gives $\text{NSPACE}(n) \subseteq \text{SPACE}(n^2)$. Therefore $A$ cannot be in NSPACE($n$), as required. □

**Nondeterministic logspace.** The class NL consists of languages accepted by nondeterministic machines using $O(\log n)$ working space. For such a machine, the number of configurations is polynomial in the input length.

**Exercise 11.9.** Prove that NL ⊆ P. Use the graph whose vertices are configurations of a nondeterministic logspace machine and whose edges are its possible moves.

*Solution.* A configuration records the machine's state, input-head position, work-tape contents, and work-tape-head position. With $O(\log n)$ work space, each part has only polynomially many possibilities, so the configuration graph has polynomial size. Its edges are possible nondeterministic moves and can be generated in polynomial time. A deterministic breadth-first search tests whether an accepting configuration is reachable from the start configuration. The search takes polynomial time and accepts exactly when some branch accepts. Hence NL ⊆ P. □

**Exercise 11.10.** Prove that NL is closed under union and intersection.

*Solution.* For the union, a nondeterministic machine chooses whether to simulate the machine for $A$ or the machine for $B$. It accepts exactly when at least one chosen simulation has an accepting branch.

For the intersection, simulate the machine for $A$ and then the machine for $B$, reusing the same work tape. The combined machine has an accepting branch exactly when both machines have accepting branches. The two simulations use $O(\log n)$ space, so both closure claims follow. □

**Exercise 11.11.** Prove that DIRECTED-REACHABILITY is in NL. Here the input is a directed graph $G$ and two vertices $s$ and $t$, and the question is whether there is a directed path from $s$ to $t$. The lecture also states that this problem is NL-complete under logspace reductions. Explain why a deterministic logspace algorithm for it would imply L = NL, and whether we know that this equality holds.

*Solution.* Store the current vertex, initially $s$, and a counter. Nondeterministically guess an outgoing edge, update the current vertex, and accept if it is $t$. Stop after at most $|V|$ steps. If there is a path from $s$ to $t$, there is one with no repeated vertices and therefore at most $|V| - 1$ edges. The current vertex and the counter each use $O(\log |V|)$ bits, and checking a guessed edge takes logarithmic space. Thus this is an NL decider.

If the problem also has a deterministic logspace decider, it is in L. By NL-completeness, every language in NL reduces to it in logspace. Exercise 11.1(b) then implies that every such language is in L, so NL ⊆ L. Since L ⊆ NL, the two classes are equal. We know this containment but do not know whether it is strict. □

**Exercise 11.12.** * Prove that NON-BIPARTITE, the problem of deciding whether an undirected graph is not bipartite, is in NL. Does this also prove that BIPARTITE is in NL? Briefly explain.

*Solution.* An undirected graph is not bipartite exactly when it contains an odd cycle. A nondeterministic machine guesses a starting vertex and an odd closed walk of length at most the number of vertices, checking each edge as it goes. It stores only the current vertex, the starting vertex, a counter, and the parity of the length, all in logarithmic space. An odd cycle supplies an accepting branch, and any odd closed walk contains an odd cycle. Hence NON-BIPARTITE is in NL.

This puts the complement of BIPARTITE in NL; by itself it does not put BIPARTITE in NL. That conclusion would require a separate result showing that NL is closed under complement. □

**From Exercise sheets 9 & 10.**

**Exercise 11.13.** * A nondeterministic Turing machine may have several possible transitions for a state and observed symbol. It accepts an input if some computation halts in the accepting state, and rejects if all computations reject. It runs in polynomial time if every computation halts within a polynomial number of steps.

Prove that $L \in \mathrm{NP}$ if and only if a polynomial-time nondeterministic Turing machine decides $L$.

*Solution.* Suppose $L \in \mathrm{NP}$, with verifier $V$, certificate bound $p(n)$, and polynomial running-time bound $q$ for $V$. A nondeterministic machine first guesses a word $s$ of length at most $p(n)$: at each step it chooses either to stop guessing or to append a letter, and it must stop after $p(n)$ letters. It then runs $V(w, s)$.

Every branch takes polynomial time: generating $s$ takes polynomial time, and verification takes at most $q(O(n + p(n)))$ steps. There is an accepting branch exactly when there is an accepted certificate for $w$. Thus this machine decides $L$ in nondeterministic polynomial time.

Conversely, let $M$ decide $L$ in at most $p(n)$ steps on every branch. Let $b$ bound the number of available transitions in any state and tape symbol. A certificate records a sequence of at most $p(n)$ transition choices, each using $O(\log(b + 1))$ bits. Since $M$ is fixed, $b$ is constant.

A deterministic verifier simulates $M$ on $w$, following the recorded choices. It rejects an unavailable choice or a branch that fails to accept within $p(n)$ steps, and accepts exactly when the recorded branch accepts. Simulating this fixed machine for polynomially many steps takes polynomial time, and the certificate has length $O(p(n))$. An accepted certificate exists exactly when $M$ has an accepting branch. Hence $L \in \mathrm{NP}$. □

**Exercise 11.14.** * Let NEXP be the class of languages decided by exponential-time nondeterministic Turing machines. Prove that if P = NP, then EXP = NEXP.

*Solution.* Clearly EXP $\subseteq$ NEXP: a deterministic machine is a nondeterministic machine with only one available transition at each step. We prove the reverse inclusion under the assumption P = NP.

Consider the language

$$B = \{\langle M, w, 1^t\rangle \mid M \text{ has an accepting computation on } w \text{ of at most } t \text{ steps}\}.$$

Here $M$ is a nondeterministic machine and the time bound is written in unary. A certificate records an accepting branch, as in Exercise 11.13. If the encoded input length is $N$, then $t \leq N$, and a transition choice can be recorded with $O(\log(N+1))$ bits. Thus the certificate has polynomial length. A universal deterministic verifier can check the $t$ steps by scanning the encoded transition table and the simulated tape; both have polynomial length. Therefore $B \in$ NP.

Assuming P = NP, fix a deterministic decider for $B$ running in time $O(N^d)$, for some constant $d$. Now let $L \in$ NEXP, with a fixed nondeterministic machine $M$ and time bound $2^{p(n)}$, where $p$ is a polynomial. On input $w$ of length $n$, construct

$$\langle M, w, 1^{2^{p(n)}}\rangle$$

and run the decider for $B$. This input has length $N = O(n + 2^{p(n)})$. Writing it takes exponential time, and the decider takes

$$O(N^d) = 2^{n^{O(1)}}$$

time as well. It accepts exactly when $M$ has an accepting branch on $w$. Hence $L \in$ EXP, giving NEXP $\subseteq$ EXP and therefore EXP = NEXP. □

**Sources.** The balanced-parentheses and closure exercises are adapted from problem set 8 in the supplied `cs-hse-toc` materials. The quantified-formula, bounded-space acceptance, and space hierarchy exercises come from problem set 7; the padding exercise comes from problem set 6.